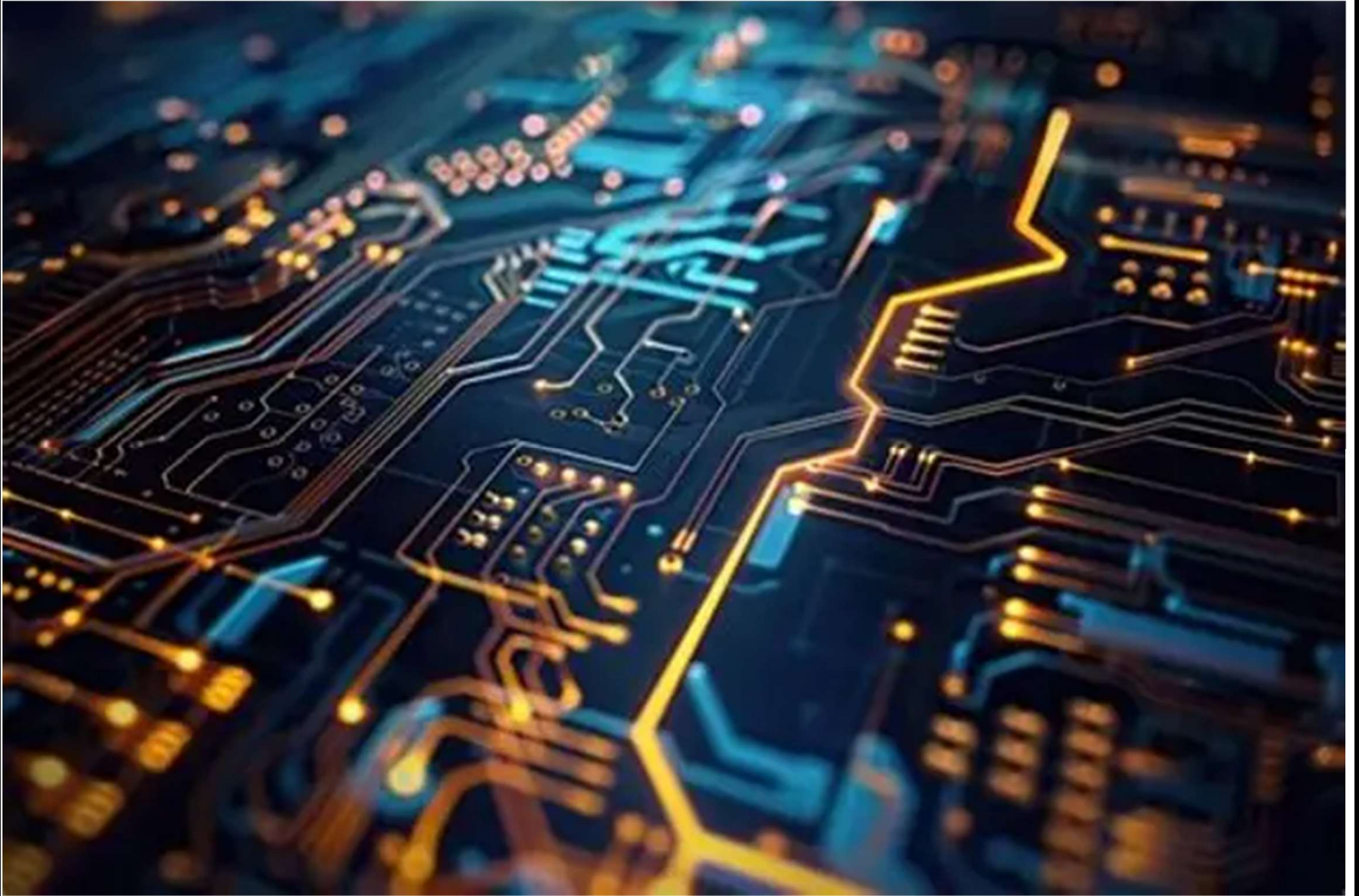


Assignment: Light Chaser



Name: Ahmed Wael Sadek El
Sawy

Assignment: Light Chaser

Clock Divider

clk_divider.v

```
module clk_divider #(
    parameter TOGGLE_COUNT = 3125000
)(//each 3125000 cycles of clk_in, clk_out toggles once ,so 8 cycles of clk_out =
6250000 cycles of clk_in, so 50,000,000 / 6250000 = 8Hz
    input wire clk_in,
    input wire reset_n,
    output reg clk_out
);
    reg [$clog2(TOGGLE_COUNT)-1:0] counter;

    always @(posedge clk_in or negedge reset_n) begin
        if (!reset_n) begin
            counter <= 0;
            clk_out <= 1'b0;
        end else begin
            if (counter == TOGGLE_COUNT - 1) begin
                counter <= 0;
                clk_out <= ~clk_out;
            end else begin
                counter <= counter + 1;
            end
        end
    end
endmodule
```

tb_clk_divider.v

```
module tb_clk_divider;
    reg clk_in, reset_n;
    wire clk_out;

    // Use a small TOGGLE_COUNT (e.g., 4) for quick simulation
    parameter T_COUNT = 4;
    integer errors = 0;
    integer i;

    clk_divider #(.TOGGLE_COUNT(T_COUNT)) DUT (
        .clk_in(clk_in), .reset_n(reset_n), .clk_out(clk_out)
    );

    initial begin
```

Assignment: Light Chaser

```
    clk_in = 0;
    forever #5 clk_in = ~clk_in;
end

initial begin
    reset_n = 0;
    @(negedge clk_in);
    reset_n = 1;

    // Synchronize to the first negative edge of the output clock
    @(negedge clk_out);

    // Self-Checking Loop
    for (i = 0; i < 3; i = i + 1) begin
        // Wait for exactly T_COUNT rising edges (when the RTL counter
increments)
        repeat(T_COUNT) @(posedge clk_in);

        // Wait half a cycle for the negedge to safely sample the updated
value
        @(negedge clk_in);

        // The output clock should have toggled to HIGH
        if (clk_out !== 1'b1) begin
            $display("ERROR: clk_out did not toggle HIGH after %0d cycles.",
T_COUNT);
            errors = errors + 1;
        end

        // Wait for another T_COUNT rising edges
        repeat(T_COUNT) @(posedge clk_in);

        // Wait half a cycle for the negedge to safely sample
        @(negedge clk_in);

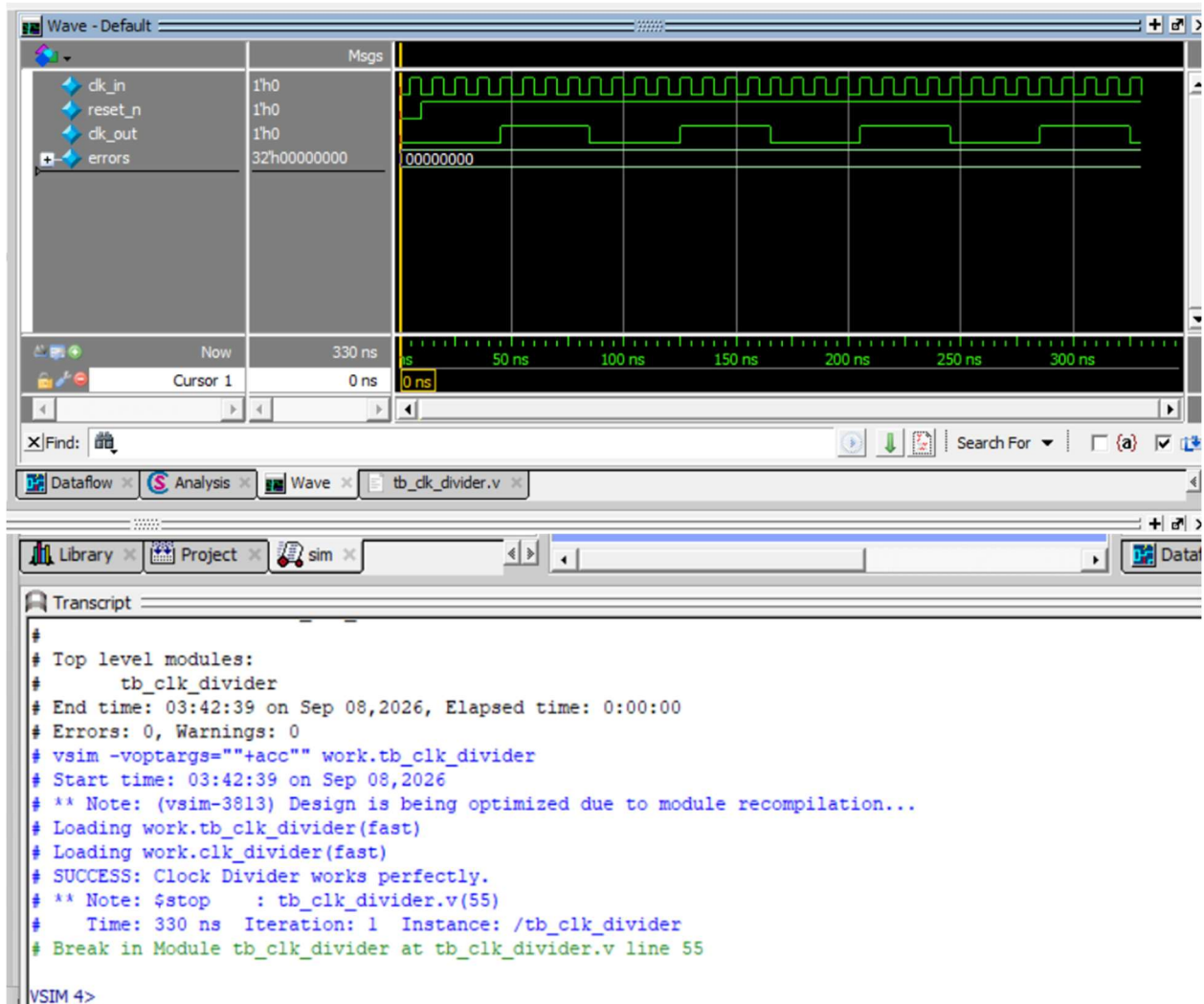
        // The output clock should have toggled to LOW
        if (clk_out !== 1'b0) begin
            $display("ERROR: clk_out did not toggle LOW after %0d cycles.",
T_COUNT);
            errors = errors + 1;
        end
    end
end

if (errors == 0) $display("SUCCESS: Clock Divider works perfectly.");
$stop;
```

Assignment: Light Chaser

```
end  
endmodule
```

Testing clk divider:



Assignment: Light Chaser

Shift Register

shift_register.v

```
module shift_register #(
    parameter NUM_BITS = 10
)(
    input wire clk,
    input wire reset_n,
    input wire hold_n,
    output reg [NUM_BITS-1:0] shift_out
);
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            shift_out <= {1'b1, {(NUM_BITS-1){1'b0}}};
        end else begin
            if (hold_n == 1'b1) begin
                shift_out <= {shift_out[0], shift_out[NUM_BITS-1:1]};
            end
        end
    end
endmodule
```

tb_shift_register.v

```
module tb_shift_register;
    parameter N_BITS = 10;

    reg clk, reset_n, hold_n;
    wire [N_BITS-1:0] shift_out;

    reg [N_BITS-1:0] expected_val;
    integer errors = 0;
    integer i;
    reg rand_hold;

    shift_register #(.NUM_BITS(N_BITS)) DUT (
        .clk(clk), .reset_n(reset_n), .hold_n(hold_n), .shift_out(shift_out)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
endmodule
```

Assignment: Light Chaser

```
initial begin
    // Initialization
    reset_n = 0; hold_n = 1;
    @(negedge clk);
    reset_n = 1;

    expected_val = {1'b1, {(N_BITS-1){1'b0}}};

    // Self-Checking Loop with Random Stimulus
    for (i = 0; i < 20; i = i + 1) begin
        // Generate random 1-bit hold_n constraint at negedge
        rand_hold = $random % 2;
        hold_n = rand_hold;

        @(negedge clk); // Step forward one full clock cycle (past the
posedge)

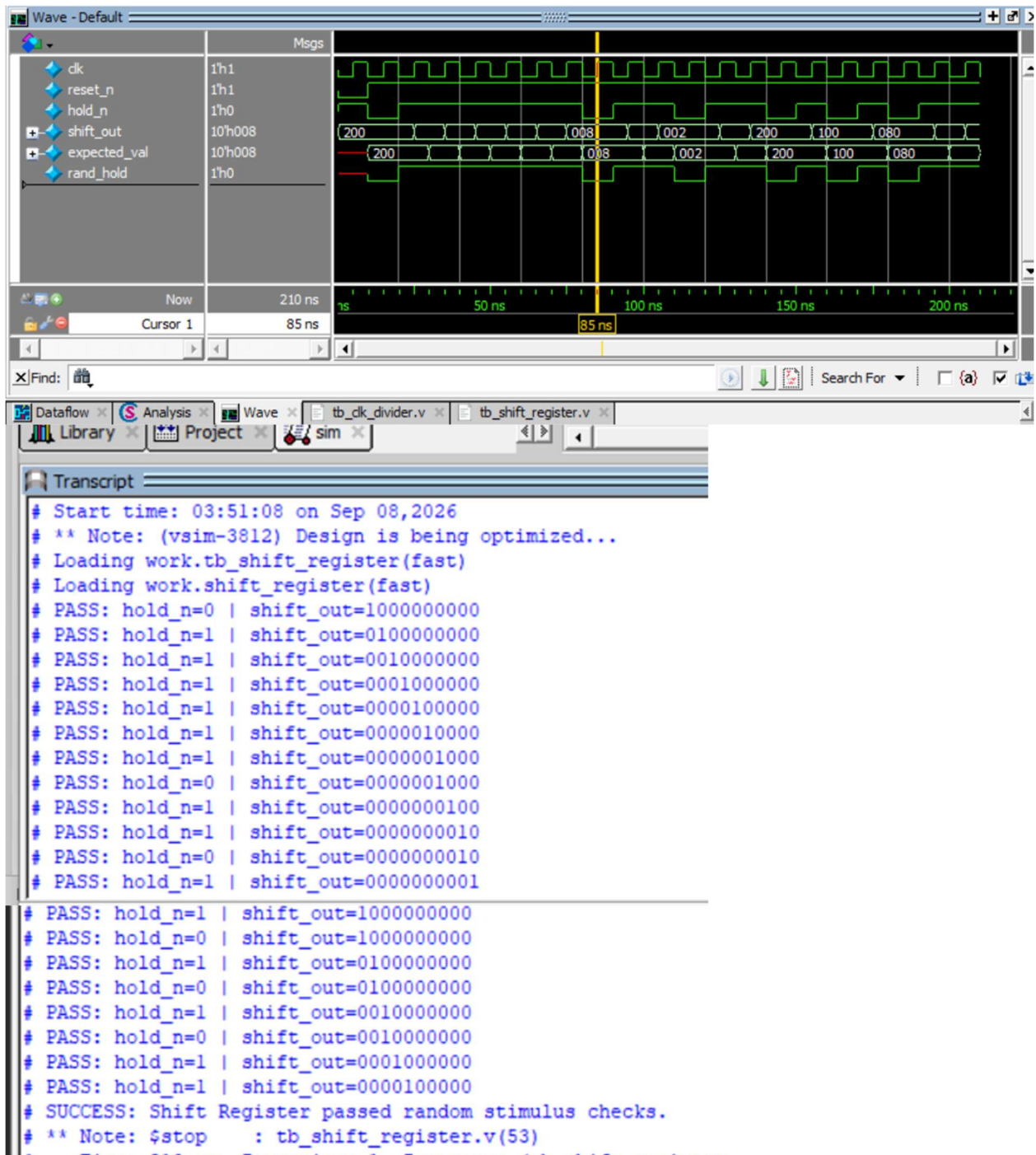
        // Update expected value if hold_n was inactive (1)
        if (rand_hold == 1'b1) begin
            expected_val = {expected_val[0], expected_val[N_BITS-1:1]};
        end

        // Validate
        if (shift_out != expected_val) begin
            $display("ERROR at iteration %0d: Expected %b, Got %b
(hold_n=%b)",
                    i, expected_val, shift_out, hold_n);
            errors = errors + 1;
        end else begin
            $display("PASS: hold_n=%b | shift_out=%b", hold_n, shift_out);
        end
    end

    if (errors == 0) $display("SUCCESS: Shift Register passed random stimulus
checks.");
    $stop;
end
endmodule
```

Assignment: Light Chaser

Testing Shift register:



Assignment: Light Chaser

Light Chaser(top module)

light_chaser.v

```
module light_chaser #(
    parameter TOGGLE_COUNT = 3125000,
    parameter NUM_BITS      = 10
)(
    input  wire          clk_50M,
    input  wire          reset_n,
    input  wire          hold_n,
    output wire [NUM_BITS-1:0] shift_out
);
    wire clk_8Hz; // Internal divided clock

    // Instantiate Clock Divider
    clk_divider #(
        .TOGGLE_COUNT(TOGGLE_COUNT)
    ) div_inst (
        .clk_in(clk_50M),
        .reset_n(reset_n),
        .clk_out(clk_8Hz)
    );

    // Instantiate Shift Register
    shift_register #(
        .NUM_BITS(NUM_BITS)
    ) sr_inst (
        .clk(clk_8Hz),
        .reset_n(reset_n),
        .hold_n(hold_n),
        .shift_out(shift_out)
    );
endmodule
```

tb_light_chaser.v

```
module tb_light_chaser;
    // Scale parameters down drastically to make simulation verifiable without
    // waiting millions of ticks
    parameter T_COUNT = 2;
    parameter N_BITS = 4;
```

Assignment: Light Chaser

```
reg clk_50M, reset_n, hold_n;
wire [N_BITS-1:0] shift_out;

reg [N_BITS-1:0] expected_val;
integer errors = 0;
integer i;

light_chaser #(.TOGGLE_COUNT(T_COUNT), .NUM_BITS(N_BITS)) DUT (
    .clk_50M(clk_50M), .reset_n(reset_n), .hold_n(hold_n),
    .shift_out(shift_out)
);

initial begin
    clk_50M = 0;
    forever #5 clk_50M = ~clk_50M;
end

initial begin
    // Initialization
    reset_n = 0; hold_n = 1;
    @(negedge clk_50M);
    reset_n = 1;
    expected_val = 4'b1000;

    // Test normal rotation
    for (i = 0; i < 4; i = i + 1) begin

        repeat(T_COUNT * 2) @(negedge clk_50M);

        expected_val = {expected_val[0], expected_val[N_BITS-1:1]};

        if (shift_out != expected_val) begin
            $display("ERROR: Expected %b, Got %b", expected_val, shift_out);
            errors = errors + 1;
        end else begin
            $display("PASS: shift_out=%b", shift_out);
        end
    end

    // Test hold functionality
    @(negedge clk_50M);
    hold_n = 0;

    repeat(T_COUNT * 4) @(negedge clk_50M);
```

Assignment: Light Chaser

```
if (shift_out != expected_val) begin
    $display("ERROR: Value shifted while hold_n was active (0).");
    errors = errors + 1;
end else begin
    $display("PASS: hold_n prevents shifting correctly.");
end

if (errors == 0) $display("SUCCESS: Top-level Light Chaser works
seamlessly.");
$stop;
end
endmodule
```

Test Light chaser:

